

Graphs

Chapter 10

With Question/Answer Animations

Graphs and Graph Models

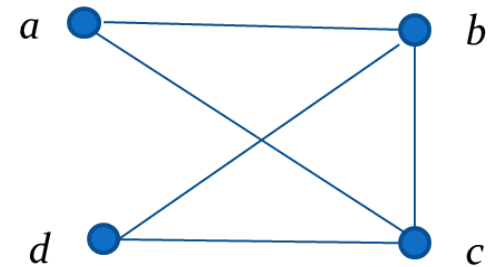
Section 10.1

Graphs

Definition: A *graph* $G = (V, E)$ consists of a nonempty set V of *vertices* (or *nodes*) and a set E of *edges*. Each edge has either one or two vertices associated with it, called its *endpoints*. An edge is said to *connect* its endpoints.

Example:

This is a graph with four vertices and five edges.



Remarks:

- The graphs we study here are unrelated to graphs of functions studied in Chapter 2.
- We have a lot of freedom when we draw a picture of a graph. *All that matters is the connections made by the edges*, not the particular geometry depicted. For example, the lengths of edges, whether edges cross, how vertices are depicted, and so on, do not matter
- A graph with an infinite vertex set is called an *infinite graph*. A graph with a finite vertex set is called a *finite graph*. We (following the text) restrict our attention to finite graphs.

Some Terminology₁

In a *simple graph* each edge connects two different vertices and no two edges connect the same pair of vertices.

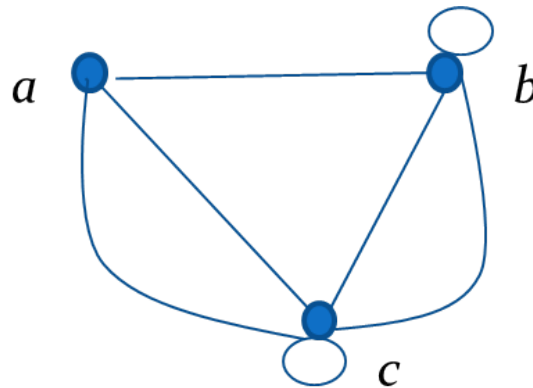
Multigraphs(多重图) may have multiple edges connecting the same two vertices. When m different edges connect the vertices u and v , we say that $\{u,v\}$ is an edge of *multiplicity* m .

An edge that connects a vertex to itself is called a *loop*圈环.

A *pseudograph*(伪图) may include loops, as well as multiple edges connecting the same pair of vertices.

Example:

This pseudograph has both multiple edges and a loop.



Remark: There is no standard terminology for graph theory. So, it is crucial that you understand the terminology being used whenever you read material about graphs.

Directed Graphs₁

Definition: An *directed graph* (有向图) (or *digraph*) $G = (V, E)$ consists of a nonempty set V of *vertices* (or *nodes*) and a set E of *directed edges* (or *arcs*). Each edge is associated with an *ordered pair* of vertices. The directed edge associated with the ordered pair (u, v) is said to *start at* u and *end at* v .

Remark:

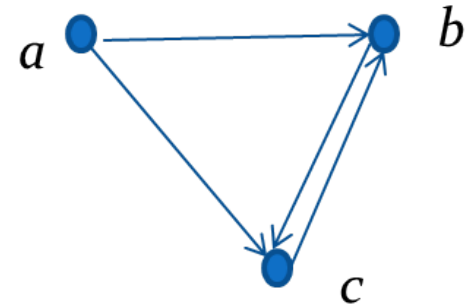
- Graphs where the end points of an edge are *not ordered* are said to be *undirected graphs* (无向图).

Some Terminology₂

A *simple directed graph* 简单有向图 has no loops and no multiple edges.

Example:

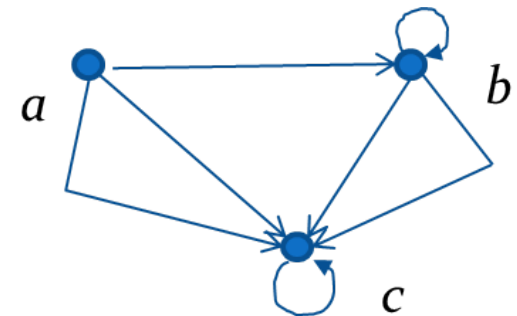
This is a directed graph with three vertices and four edges.



A *directed multigraph* 有向多重图 may have multiple directed edges. When there are m directed edges from the vertex u to the vertex v , we say that (u,v) is an edge of *multiplicity* m .

Example:

In this directed multigraph the multiplicity of (a,b) is 1 and the multiplicity of (b,c) is 2.



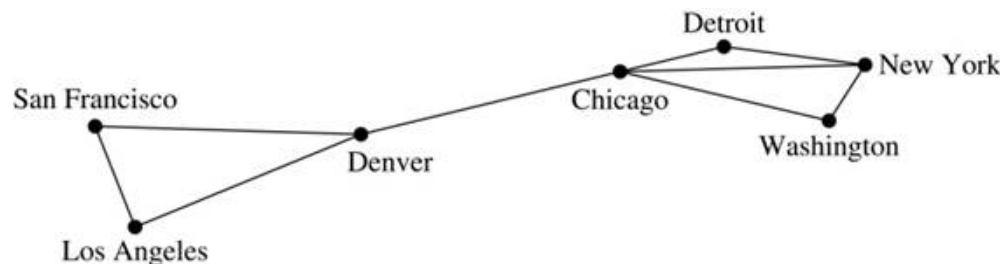
Graph Models:

Computer Networks₁

When we build a graph model, we use the appropriate type of graph to capture the important features of the application.

We illustrate this process using graph models of different types of computer networks. In all these graph models, the vertices represent data centers and the edges represent communication links.

To model a computer network where we are only concerned **whether two data centers are connected** by a communications link, we use a **simple graph**. This is the appropriate type of graph when we only care whether two data centers are directly linked (and not how many links there may be) and all communications links work in both directions.

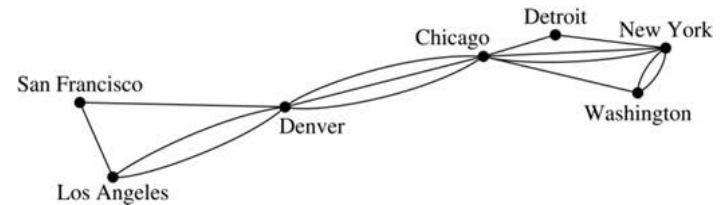


[Jump to long description](#)

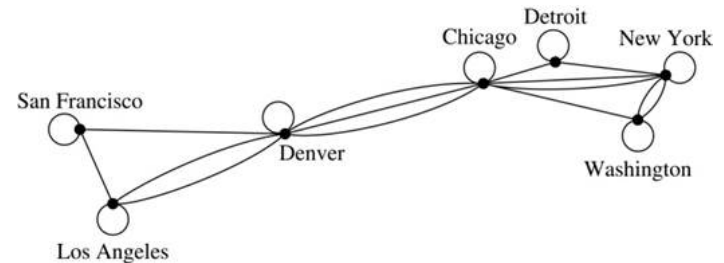
Graph Models:

Computer Networks₂

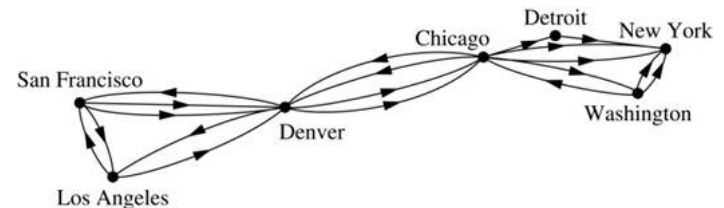
To model a computer network where we care about **the number of links** between data centers, we use a **multigraph**.



To model a computer network with **diagnostic links** at data centers, we use a **pseudograph**, as loops are needed.



To model a network with **multiple one-way links** 多个单向链接, we use a **directed multigraph**. Note that we could use a directed graph without multiple edges if we only care whether there is at least one link from a data center to another data center.



[Jump to long description](#)

Graph Terminology: Summary

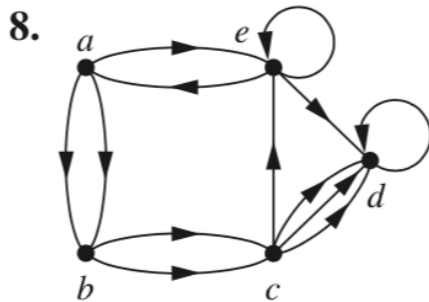
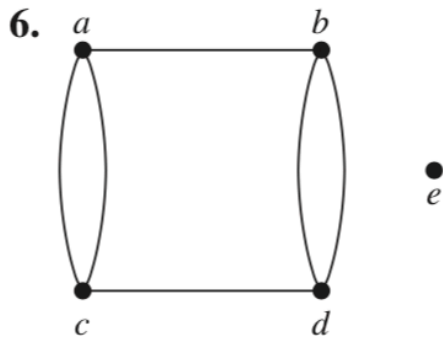
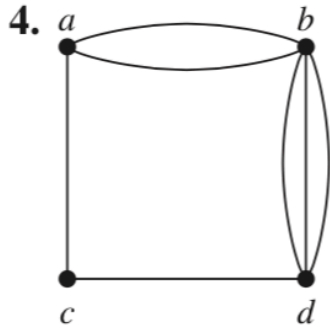
To understand the structure of a graph and to build a graph model, we ask these questions:

- Are the edges of the graph undirected or directed (or both)?
- If the edges are undirected, are multiple edges present that connect the same pair of vertices? If the edges are directed, are multiple directed edges present?
- Are loops present?

TABLE 1 Graph Terminology.

<i>Type</i>	<i>Edges</i>	<i>Multiple Edges Allowed?</i>	<i>Loops Allowed?</i>
Simple graph	Undirected	No	No
Multigraph	Undirected	Yes	No
Pseudograph	Undirected	Yes	Yes
Simple directed graph	Directed	No	No
Directed multigraph	Directed	Yes	Yes
Mixed graph	Directed and undirected	Yes	Yes

For Exercises 3–9, determine whether the graph shown has directed or undirected edges, whether it has multiple edges, and whether it has one or more loops. Use your answers to determine the type of graph in Table 1 this graph is.



Graph Terminology and Special Types of Graphs

Section 10.2

10.2.2 Basic Terminology

Definition 1. Two vertices u, v in an undirected graph G are called *adjacent* 相邻的 (or *neighbors*) in G if there is an edge e between u and v . Such an edge e is called *incident* (关联) with the vertices u and v and e is said to *connect* u and v .

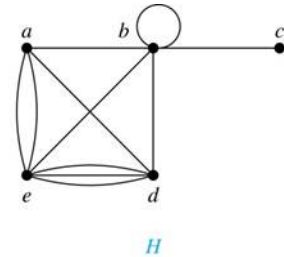
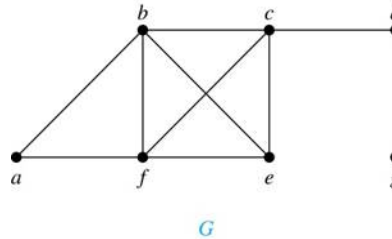
Definition 2. The set of all neighbors of a vertex v of $G = (V, E)$, denoted by $N(v)$, is called the *neighborhood of v* (v 的邻域). If A is a subset of V , we denote by $N(A)$ the set of all vertices in G that are adjacent to at least one vertex in A . So, $N(A) = \bigcup_{v \in A} N(v)$.

Definition 3. The *degree of a vertex in a undirected graph* 度数 is the number of edges incident with it, except that a loop at a vertex contributes two to the degree of that vertex. The degree of the vertex v is denoted by $\deg(v)$.

Degrees and Neighborhoods of Vertices

Example: What are the degrees and neighborhoods of the vertices in the graphs G and H ?

Solution:



G : $\deg(a) = 2$, $\deg(b) = \deg(c) = \deg(f) = 4$, $\deg(d) = 1$,
 $\deg(e) = 3$, $\deg(g) = 0$.

$N(a) = \{b, f\}$, $N(b) = \{a, c, e, f\}$, $N(c) = \{b, d, e, f\}$, $N(d) = \{c\}$,

$N(e) = \{b, c, f\}$, $N(f) = \{a, b, c, e\}$, $N(g) = \emptyset$.

H : $\deg(a) = 4$, $\deg(b) = \deg(e) = 6$, $\deg(c) = 1$, $\deg(d) = 5$.

$N(a) = \{b, d, e\}$, $N(b) = \{a, b, c, d, e\}$, $N(c) = \{b\}$,

$N(d) = \{a, b, e\}$, $N(e) = \{a, b, d\}$.

[Jump to long description](#)

Degrees of Vertices

Theorem 1 (*Handshaking Theorem*): If $G = (V, E)$ is an undirected graph with m edges, then

$$2m = \sum_{v \in V} \deg(v)$$

Proof:

Each edge contributes twice to the degree count of all vertices. Hence, both the left-hand and right-hand sides of this equation equal twice the number of edges.

Think about the graph where vertices represent the people at a party and an edge connects two people who have shaken hands.

无向图中所有顶点的度之和是边数的两倍。

Handshaking Theorem

We now give two examples illustrating the usefulness of the handshaking theorem.

Example: How many edges are there in a graph with 10 vertices of degree six?

Solution: Because the sum of the degrees of the vertices is $6 \cdot 10 = 60$, the handshaking theorem tells us that $2m = 60$. So the number of edges $m = 30$.

Example: If a graph has 5 vertices, can each vertex have degree 3?

Solution: This is not possible by the handshaking theorem, because the sum of the degrees of the vertices $3 \cdot 5 = 15$ is odd.

Degree of Vertices

Theorem 2: An undirected graph has an even number of vertices of odd degree. 在无向图里，度数为奇的顶点的数量必为偶。

Proof: Let V_1 be the vertices of even degree and V_2 be the vertices of odd degree in an undirected graph $G = (V, E)$ with m edges. Then

$$\text{even} \rightarrow 2m = \sum_{v \in V} \deg(v) = \sum_{v \in V_1} \deg(v) + \sum_{v \in V_2} \deg(v).$$

must be even
since $\deg(v)$ is
even for each v
 $\in V_1$

This sum must be even because $2m$ is even and the sum of the degrees of the vertices of even degrees is also even. Because this is the sum of the degrees of all vertices of odd degree in the graph, there must be an even number of such vertices.

Directed Graphs₂

Recall the definition of a directed graph.

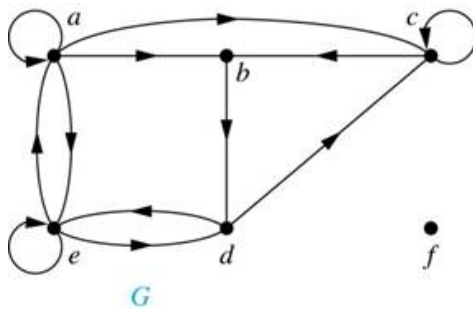
Definition: An *directed graph* $G = (V, E)$ consists of V , a nonempty set of *vertices* (or *nodes*), and E , a set of *directed edges* or *arcs*. Each edge is an ordered pair of vertices. The directed edge (u, v) is said to start at u and end at v .

Definition: Let (u, v) be an edge in G . Then u is the *initial vertex* of this edge and is *adjacent to* v and v is the *terminal (or end) vertex* of this edge and is *adjacent from* u . The initial and terminal vertices of a loop are the same.

Directed Graphs₃

Definition: The *in-degree of a vertex v* , denoted $\deg^-(v)$, is the number of edges which terminate at v . The *out-degree of v* , denoted $\deg^+(v)$, is the number of edges with v as their initial vertex. Note that a loop at a vertex contributes 1 to both the in-degree and the out-degree of the vertex.

Example: In the graph G we have



$$\deg^-(a) = 2, \deg^-(b) = 2, \deg^-(c) = 3, \deg^-(d) = 2, \\ \deg^-(e) = 3, \deg^-(f) = 0.$$

$$\deg^+(a) = 4, \deg^+(b) = 1, \deg^+(c) = 2, \deg^+(d) = 2, \\ \deg^+(e) = 3, \deg^+(f) = 0.$$

[Jump to long description](#)

Directed Graphs₄

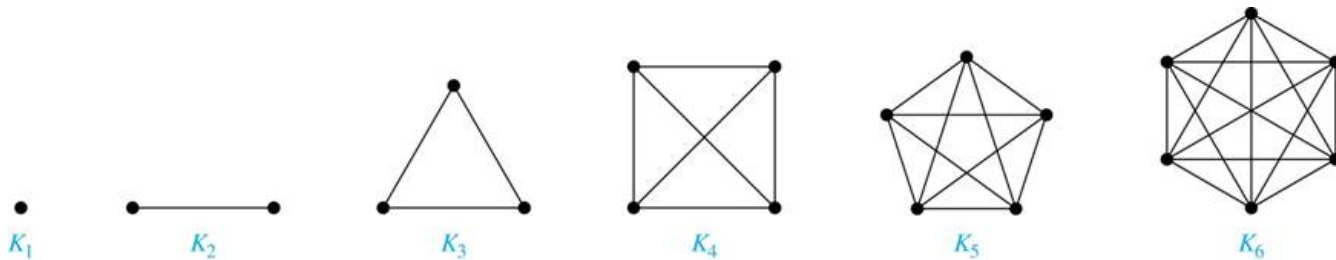
Theorem 3: Let $G = (V, E)$ be a graph with directed edges. Then:

$$|E| = \sum_{v \in V} \deg^{-}(v) = \sum_{v \in V} \deg^{+}(v).$$

Proof: The first sum counts the number of outgoing edges over all vertices and the second sum counts the number of incoming edges over all vertices. It follows that both sums equal the number of edges in the graph.

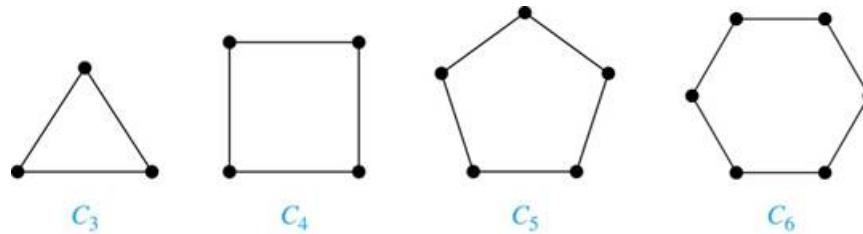
10.2.3 Special Types of Simple Graphs: Complete Graphs

A *complete graph on n vertices* 完全图, denoted by K_n , is the simple graph that contains exactly one edge between each pair of distinct vertices.

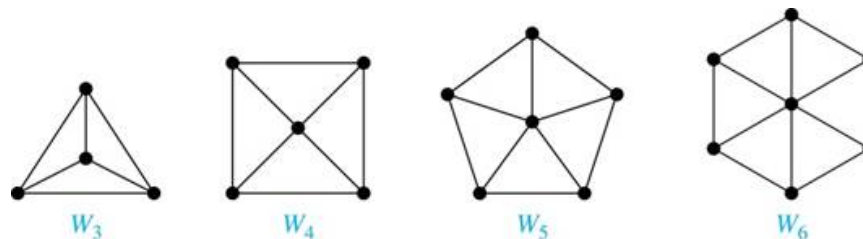


Special Types of Simple Graphs: Cycles and Wheels

A *cycle* 圈图 C_n for $n \geq 3$ consists of n vertices $v_1, v_2, \dots, v_n$, and edges $\{v_1, v_2\}, \{v_2, v_3\}, \dots, \{v_{n-1}, v_n\}, \{v_n, v_1\}$.



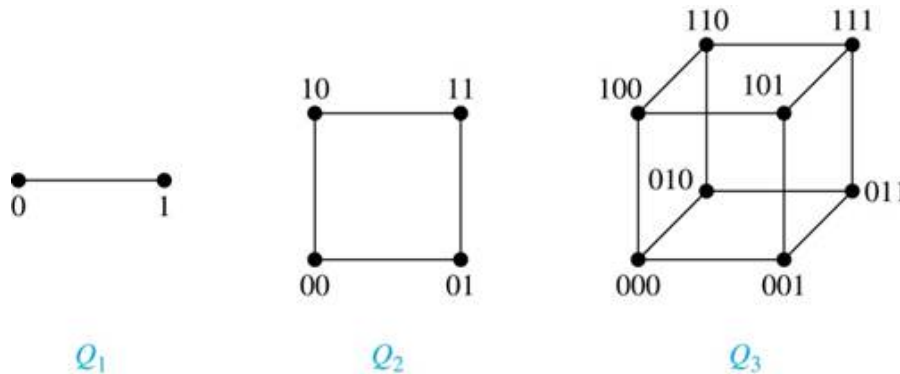
A *wheel* 轮图 W_n is obtained by adding an additional vertex to a cycle C_n for $n \geq 3$ and connecting this new vertex to each of the n vertices in C_n by new edges.



[Jump to long description](#)

Special Types of Simple Graphs: n -Cubes

An *n -dimensional hypercube* (n 维超立方体), or *n -cube*, Q_n , is a graph with 2^n vertices representing all bit strings of length n , where there is an edge between two vertices that differ in exactly one bit position.



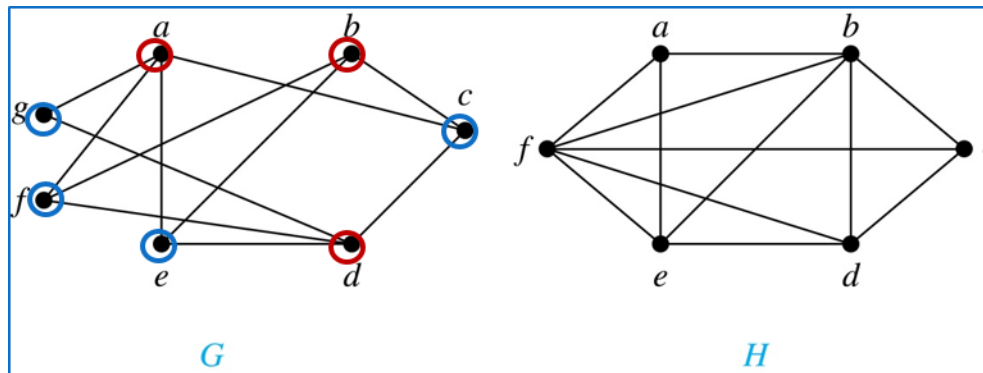
[Jump to long description](#)

10.2.4 Bipartite Graphs 偶图, 二分图

Definition: A simple graph G is *bipartite* if V can be partitioned into two disjoint subsets V_1 and V_2 such that every edge connects a vertex in V_1 and a vertex in V_2 . In other words, there are no edges which connect two vertices in V_1 or in V_2 .

It is not hard to show that an equivalent definition of a bipartite graph is a graph where it is possible to color the vertices red or blue so that no two adjacent vertices are the same color.

G is bipartite

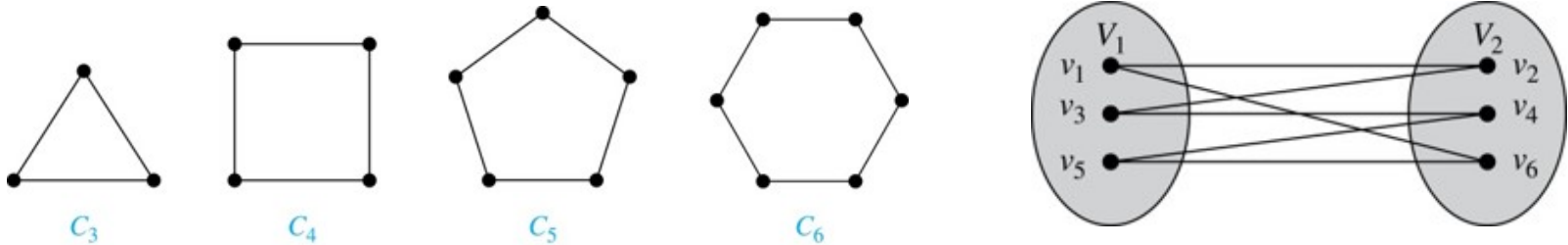


H is not bipartite since if we color a red, then the adjacent vertices f and b must both be blue.

Bipartite Graphs₂

Example: Show that C_6 is bipartite.

Solution: We can partition the vertex set into $V_1 = \{v_1, v_3, v_5\}$ and $V_2 = \{v_2, v_4, v_6\}$ so that every edge of C_6 connects a vertex in V_1 and V_2 .



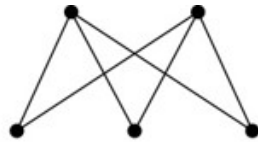
Example: Show that C_3 is not bipartite.

Solution: If we divide the vertex set of C_3 into two nonempty sets, one of the two must contain two vertices. But in C_3 every vertex is connected to every other vertex. Therefore, the two vertices in the same partition are connected. Hence, C_3 is not bipartite.

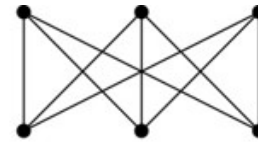
Complete Bipartite Graphs

Example: A *complete bipartite graph* $K_{m,n}$ 完全二分图 is a graph that has its vertex set partitioned into two subsets V_1 of size m and V_2 of size n such that there is an edge from every vertex in V_1 to every vertex in V_2 .

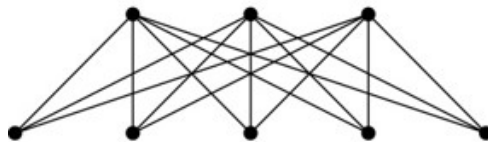
We display four complete bipartite graphs here.



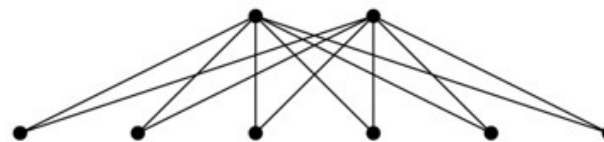
$K_{2,3}$



$K_{3,3}$



$K_{3,5}$



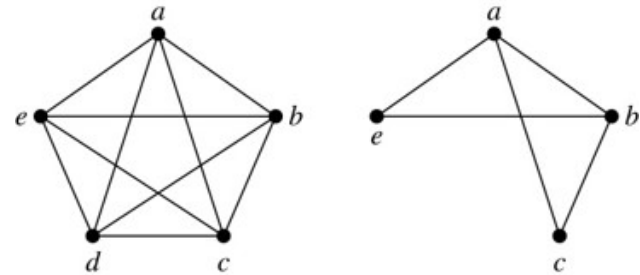
$K_{2,6}$

[Jump to long description](#)

10.2.7 New Graphs from Old₁

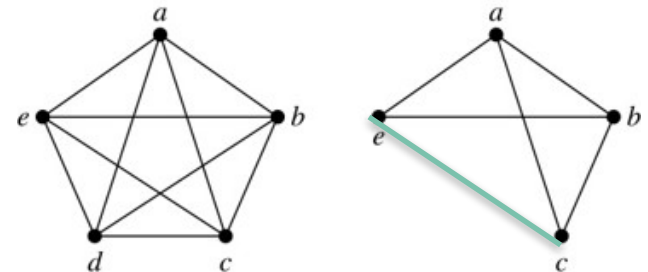
Definition: A *subgraph of a graph* $G = (V, E)$ (图的子图) is a graph (W, F) , where $W \subset V$ and $F \subset E$. A subgraph H of G is a proper subgraph of G if $H \neq G$.

Example: Here we show K_5 and one of its subgraphs.



Definition: Let $G = (V, E)$ be a simple graph. The *subgraph induced by a subset W* of the vertex set V (由 V 的子集 W 导出的子图) is the graph (W, F) , where the edge set F contains an edge in E if and only if both endpoints are in W .

Example: Here we show K_5 . If we add the edge connecting c and e to G , we obtain the subgraph induced by $W = \{a, b, c, e\}$.

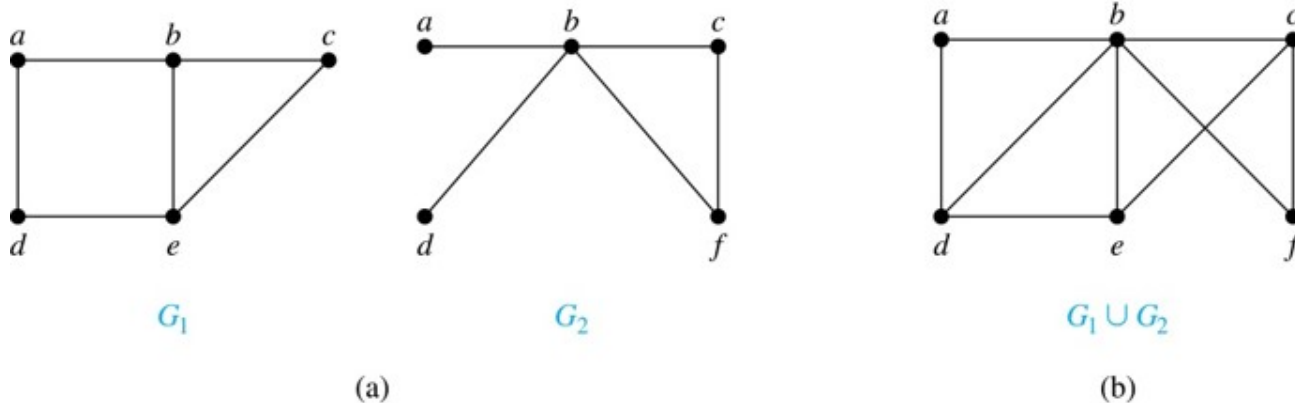


[Jump to long description](#)

New Graphs from Old₂

Definition: The *union* of two simple graphs $G_1 = (V_1, E_1)$ and $G_2 = (V_2, E_2)$ is the simple graph with vertex set $V_1 \cup V_2$ and edge set $E_1 \cup E_2$. The union of G_1 and G_2 is denoted by $G_1 \cup G_2$.

Example:



[Jump to long description](#)

Summary

1, Basic Terminology

Definition 1. Two vertices u, v in an undirected graph G are called *adjacent* 相邻的 (or *neighbors*) in G if there is an edge e between u and v . Such an edge e is called *incident* (关联) *with the vertices u and v* and e is said to *connect u and v* .

Definition 2. The set of all neighbors of a vertex v of $G = (V, E)$, denoted by $N(v)$, is called the *neighborhood of v* . If A is a subset of V , we denote by $N(A)$ the set of all vertices in G that are adjacent to at least one vertex in A . So, $N(A) = \bigcup_{v \in A} N(v)$.

Definition 3. The *degree of a vertex in a undirected graph* 度数 is the number of edges incident with it, except that a loop at a vertex contributes two to the degree of that vertex. The degree of the vertex v is denoted by $\deg(v)$.

Summary

Theorem 1 (*Handshaking Theorem*):): If $G = (V, E)$ is an undirected graph with m edges, then $2m = \sum_{v \in V} \deg(v)$

Theorem 2: An undirected graph has an even number of vertices of odd degree.

Directed Graph

Definition: The *in-degree of a vertex v* , denoted $\deg^-(v)$, is the number of edges which terminate at v . The *out-degree of v* , denoted $\deg^+(v)$, is the number of edges with v as their initial vertex.

Theorem 3: Let $G = (V, E)$ be a graph with directed edges. Then:

$$|E| = \sum_{v \in V} \deg^-(v) = \sum_{v \in V} \deg^+(v).$$

Section Summary,

2, Some Special Types of Graphs

A **complete graph on n vertices** 完全图, denoted by K_n

A **cycle** 圈图 C_n for $n \geq 3$

A **wheel** 轮图 W_n

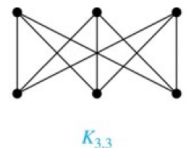
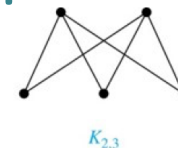
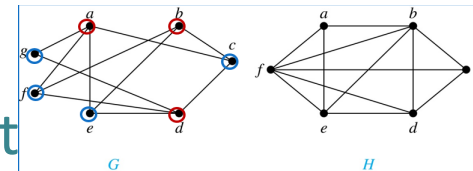
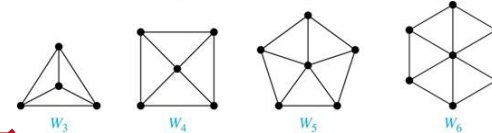
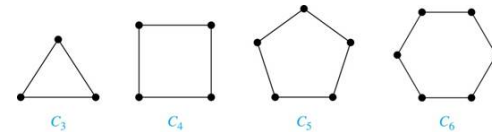
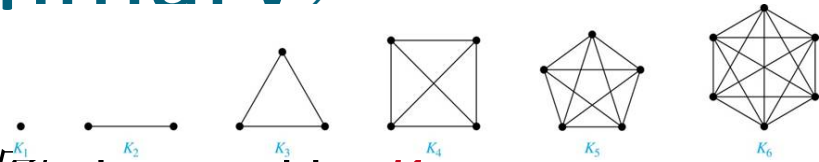
An **n -dimensional hypercube** (n 维超立方体), or **n -cube**, Q_n

3, Bipartite Graphs

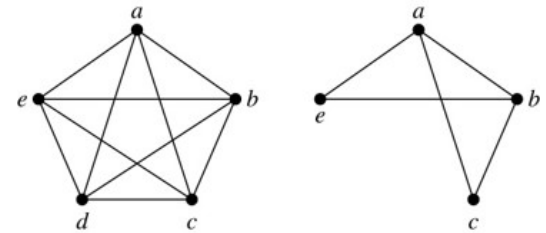
Definition: A simple graph G is **bipartite** if V can be partitioned into two disjoint subsets V_1 and V_2 such that every edge connects a vertex in V_1 and a vertex in V_2 . In other words, there are no edges which connect two vertices in V_1 or in V_2 .

It is possible to color the vertices red or blue so that no two adjacent vertices are the same color.

A **complete bipartite graph** $K_{m,n}$



Summary

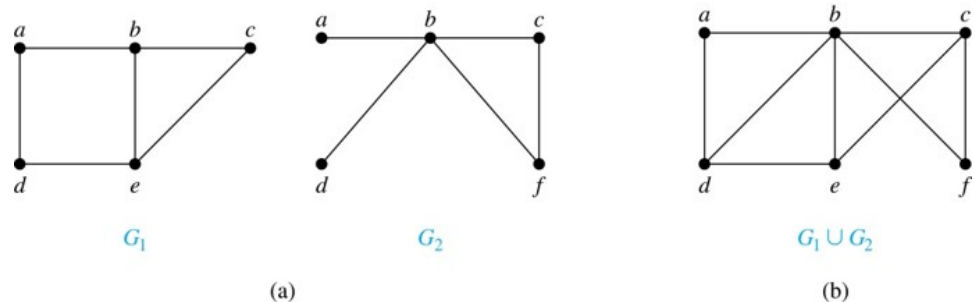


4, New Graphs from Old

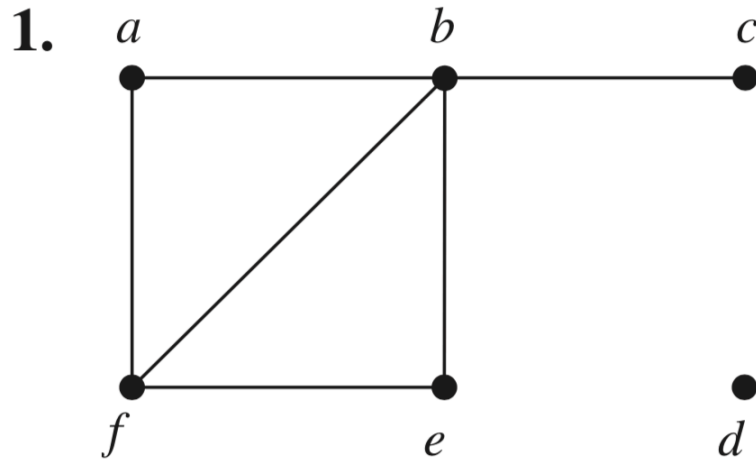
Definition: A *subgraph of a graph* $G = (V, E)$ (图的子图) is a graph (W, F) , where $W \subset V$ and $F \subset E$.

Definition: Let $G = (V, E)$ be a simple graph. The *subgraph induced by a subset W* of the vertex set V (由 V 的子集 W 导出的子图) is the graph (W, F) , where the edge set F contains an edge in E if and only if both endpoints are in W .

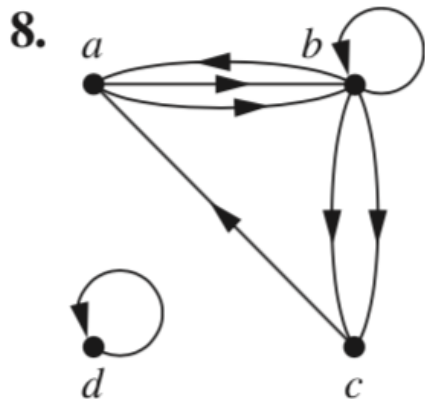
Definition: The *union* of two simple graphs $G_1 = (V_1, E_1)$ and $G_2 = (V_2, E_2)$ is the simple graph with vertex set $V_1 \cup V_2$ and edge set $E_1 \cup E_2$. The union of G_1 and G_2 is denoted by $G_1 \cup G_2$.



In Exercises 1–3 find the number of vertices, the number of edges, and the degree of each vertex in the given undirected graph. Identify all isolated and pendant vertices.



In Exercises 7–9 determine the number of vertices and edges and find the in-degree and out-degree of each vertex for the given directed multigraph.



20. Draw these graphs.

a) K_7

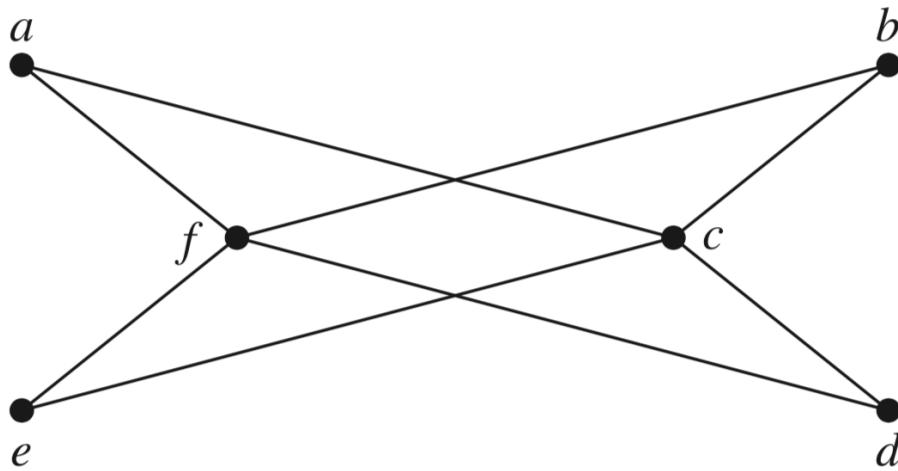
b) $K_{1,8}$

d) C_7

e) W_7

In Exercises 21–25 determine whether the graph is bipartite. You may find it useful to apply Theorem 4 and answer the question by determining whether it is possible to assign either red or blue to each vertex so that no two adjacent vertices are assigned the same color.

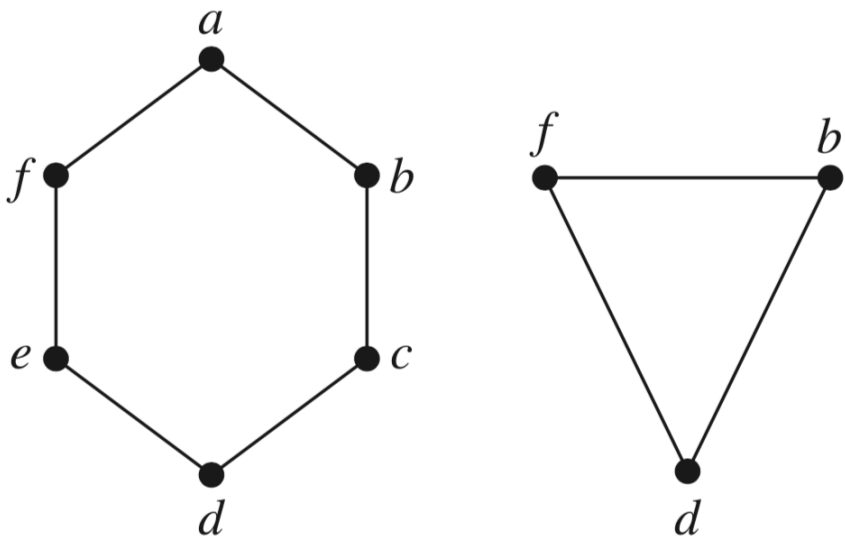
24.



42. How many edges does a graph have if its degree sequence is 4, 3, 3, 2, 2? Draw such a graph.

In Exercises 58–60 find the union of the given pair of simple graphs. (Assume edges with the same endpoints are the same.)

58.



Representing Graphs and Graph Isomorphism

Section 10.3

10.3.2 Representing Graphs: Adjacency Lists

Definition: An *adjacency list* 邻接表 can be used to represent a graph with no multiple edges by specifying the vertices that are adjacent to each vertex of the graph.

Example:

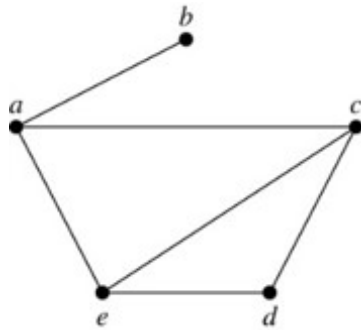


TABLE 1 An Adjacency List for a Simple Graph.

Vertex	Adjacent Vertices
a	b, c, e
b	a
c	a, d, e
d	c, e
e	a, c, d

Example:

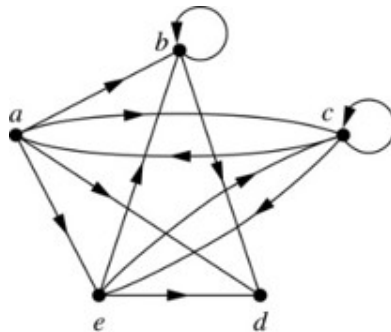


TABLE 2 An Adjacency List for a Directed Graph.

Initial Vertex	Terminal Vertices
a	b, c, d, e
b	b, d
c	a, c, e
d	
e	b, c, d

[Jump to long description](#)

10.4.3 Representation of Graphs: Adjacency Matrices₁

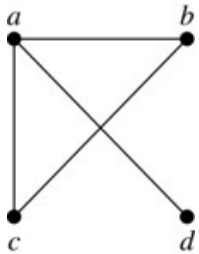
Definition: Suppose that $G = (V, E)$ is a simple graph where $|V| = n$. Arbitrarily list the vertices of G as $v_1, v_2, \dots, v_n$. The *adjacency matrix* 邻接矩阵 $\mathbf{A}_G$ of G , with respect to the listing of vertices, is the $n \times n$ zero-one matrix with 1 as its (i, j) th entry when v_i and v_j are adjacent, and 0 as its (i, j) th entry when they are not adjacent.

- In other words, if the graph's adjacency matrix is

$$\mathbf{A}_G = [a_{ij}], \text{ then } a_{ij} = \begin{cases} 1 & \text{if } \{v_i, v_j\} \text{ is an edge of } G, \\ 0 & \text{otherwise.} \end{cases}$$

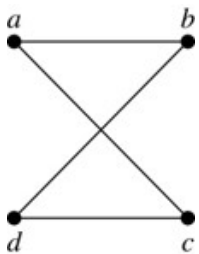
Representation of Graphs: Adjacency Matrices₂

Example:



$$\begin{bmatrix} 0 & 1 & 1 & 1 \\ 1 & 0 & 1 & 0 \\ 1 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 \end{bmatrix}$$

The ordering of vertices is a, b, c, d.



$$\begin{bmatrix} 0 & 1 & 1 & 0 \\ 1 & 0 & 0 & 1 \\ 1 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 \end{bmatrix}$$

The ordering of vertices is a, b, c, d.

When a graph is **sparse**, that is, it has few edges relatively to the total number of possible edges, it is much more efficient to represent the graph using an **adjacency list** than an adjacency matrix. But for a **dense graph**, which includes a high percentage of possible edges, an **adjacency matrix** is preferable.

Note: The adjacency matrix of a simple graph is **symmetric**, i.e., $a_{ij} = a_{ji}$

Also, since there are no loops, each diagonal entry a_{ii} for $i = 1, 2, 3, \dots, n$, is 0.

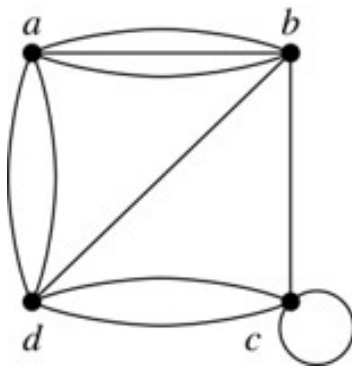
Representation of Graphs: Adjacency Matrices₃

Adjacency matrices can also be used to represent graphs with loops and multiple edges.

A loop at the vertex v_i is represented by a 1 at the (i, i) th position of the matrix.

When multiple edges connect the same pair of vertices v_i and v_j , (or if multiple loops are present at the same vertex), the (i, j) th entry equals the number of edges connecting the pair of vertices.

Example: We give the adjacency matrix of the pseudograph shown here using the ordering of vertices a, b, c, d .



$$\begin{bmatrix} 0 & 3 & 0 & 2 \\ 3 & 0 & 1 & 1 \\ 0 & 1 & 1 & 2 \\ 2 & 1 & 2 & 0 \end{bmatrix}$$

[Jump to long description](#)

Representation of Graphs: Adjacency Matrices₄

Adjacency matrices can also be used to represent **directed graphs**. The matrix for a directed graph $G = (V, E)$ has a **1** in its (i, j) th position if **there is an edge from v_i to v_j** , where $v_1, v_2, \dots, v_n$ is a list of the vertices.

- In other words, if the graph's adjacency matrix is $\mathbf{A}_G = [a_{ij}]$, then
$$a_{ij} = \begin{cases} 1 & \text{if } \{v_i, v_j\} \text{ is an edge of } G, \\ 0 & \text{otherwise.} \end{cases}$$
- The adjacency matrix for a directed graph does not have to be **symmetric**, because there may not be an edge from v_i to v_j , when there is an edge from v_j to v_i .
- To represent directed multigraphs, the value of a_{ij} is the number of edges connecting v_i to v_j .

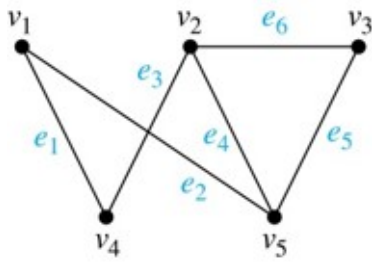
10.3.4 Representation of Graphs: Incidence Matrices₁

Definition: Let $G = (V, E)$ be an undirected graph with vertices where $v_1, v_2, \dots, v_n$ and edges $e_1, e_2, \dots, e_m$. The incidence matrix 关联矩阵 with respect to the ordering of V and E is the $n \times m$ matrix $\mathbf{M} = [m_{ij}]$, where

$$m_{ij} = \begin{cases} 1 & \text{when edge } e_j \text{ is incident with } v_i, \\ 0 & \text{otherwise.} \end{cases}$$

Representation of Graphs: Incidence Matrices₂

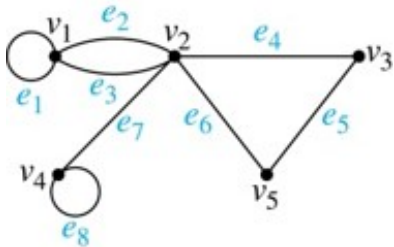
Example: Simple Graph and Incidence Matrix



$$\begin{bmatrix} 1 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 1 & 0 & 1 \\ 0 & 0 & 0 & 0 & 1 & 1 \\ 1 & 0 & 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 1 & 1 & 0 \end{bmatrix}$$

The rows going from top to bottom represent v_1 through v_5 and the columns going from left to right represent e_1 through e_6 .

Example: Pseudograph and Incidence Matrix



$$\begin{bmatrix} 1 & 1 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 & 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 0 & 1 & 1 & 0 & 0 \end{bmatrix}$$

The rows going from top to bottom represent v_1 through v_5 and the columns going from left to right represent e_1 through e_8 .

[Jump to long description](#)

Section Summary₃

1, Adjacency Lists

Definition: An **adjacency list** 邻接表 can be used to represent a graph with no multiple edges by specifying the vertices that are adjacent to each vertex of the graph.

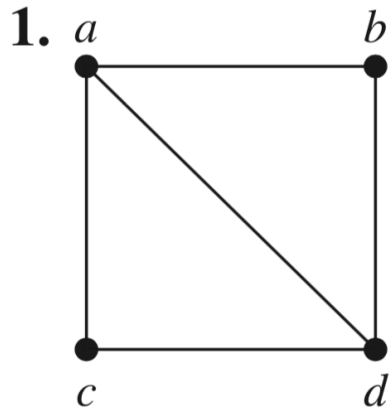
2, Adjacency Matrices

If the graph's adjacency matrix is $\mathbf{A}_G = [a_{ij}]$, then $a_{ij} = \begin{cases} 1 & \text{if } \{v_i, v_j\} \text{ is an edge of } G, \\ 0 & \text{otherwise.} \end{cases}$

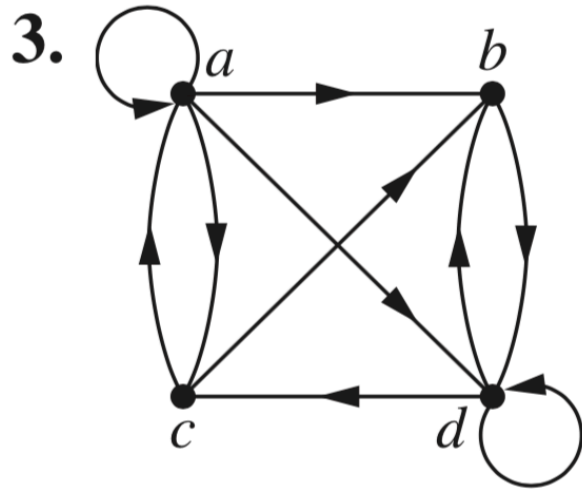
3, Incidence Matrices

The **incidence matrix** 关联矩阵 with respect to the ordering of V and E is the $n \times m$ matrix $\mathbf{M} = [m_{ij}]$, where $m_{ij} = \begin{cases} 1 & \text{when edge } e_j \text{ is incident with } v_i, \\ 0 & \text{otherwise.} \end{cases}$

Represent the graph in Exercise 1 with an adjacency list, an adjacency matrix, an incidence matrix.



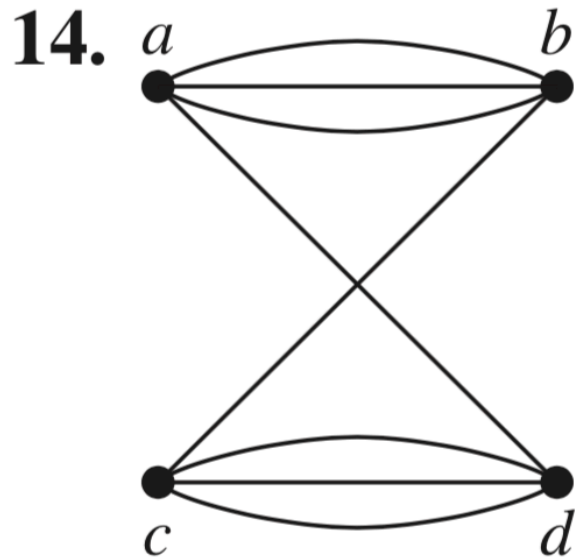
Represent the graph in Exercise 3 with an adjacency list, an adjacency matrix.



In Exercises 10–12 draw a graph with the given adjacency matrix.

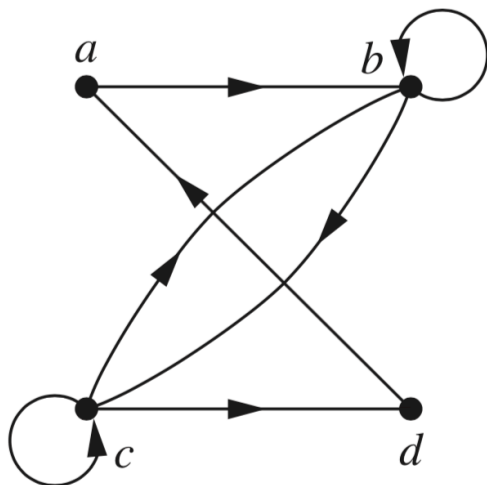
12.
$$\begin{bmatrix} 1 & 1 & 1 & 0 \\ 0 & 0 & 1 & 0 \\ 1 & 0 & 1 & 0 \\ 1 & 1 & 1 & 0 \end{bmatrix}$$

In Exercises 13–15 represent the given graph using an adjacency matrix.



In Exercises 19–21 find the adjacency matrix of the given directed multigraph with respect to the vertices listed in alphabetic order.

19.



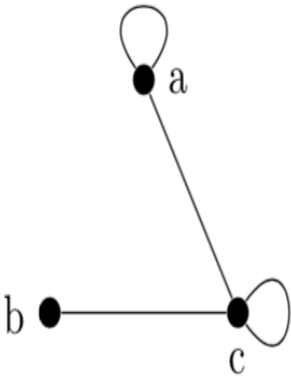
19.
$$\begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 1 & 1 & 0 \\ 0 & 1 & 1 & 1 \\ 1 & 0 & 0 & 0 \end{bmatrix}$$

In Exercises 22–24 draw the graph represented by the given adjacency matrix.

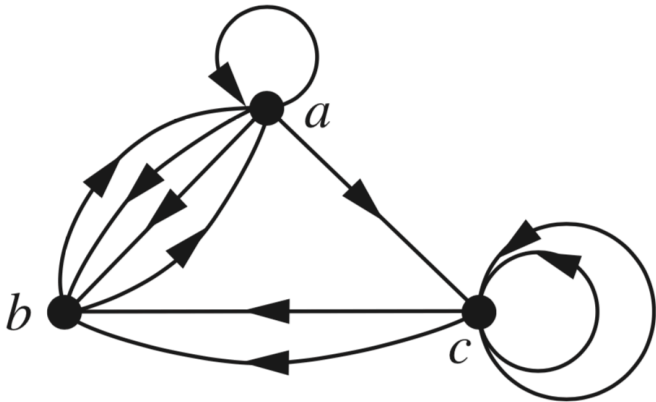
22.
$$\begin{bmatrix} 1 & 0 & 1 \\ 0 & 0 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

23.
$$\begin{bmatrix} 1 & 2 & 1 \\ 2 & 0 & 0 \\ 0 & 2 & 2 \end{bmatrix}$$

22. a) This matrix is symmetric, so we can take the graph to be undirected. No parallel edges are present, since no entries exceed 1.



23.



Homework

Page 683 Determine the type of graph (shown in Table 1) in exercise 3 5 7.

Page 699 2 7 22 35a

Draw these graphs K_5 $K_{2,5}$ C_5 W_5

Page 710 Represent the graph in Exercise 2 with an adjacency list, an adjacency matrix, an incidence matrix.

Page 710 11 15

Connectivity

Section 10.4

10.4.2 Paths₁

Informal Definition: A *path* 路径 is a sequence of edges that begins at a vertex of a graph and travels from vertex to vertex *along edges* of the graph. As the path travels along its edges, it visits the vertices along this path, that is, the endpoints of these edges.

Applications: Numerous problems can be modeled with paths formed by traveling along edges of graphs such as:

- determining whether a message can be sent between two computers.
- efficiently planning routes for mail delivery.

Paths₂

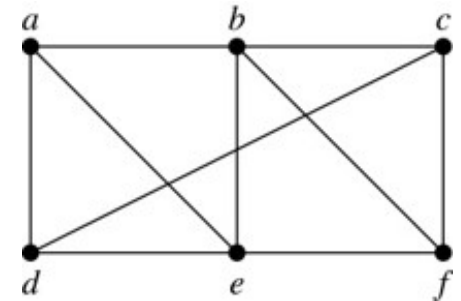
Definition: Let n be a nonnegative integer and G an undirected graph. A path of length n from u to v in G is a sequence of n edges $e_1, \dots, e_n$ of G for which there exists a sequence $x_0 = u, x_1, \dots, x_{n-1}, x_n = v$ of vertices such that e_i has, for $i = 1, \dots, n$, the endpoints x_{i-1} and x_i .

- When the graph is simple, we denote this path by its vertex sequence $x_0, x_1, \dots, x_n$ (since listing the vertices uniquely determines the path).
- The path is a circuit 回路 if it begins and ends at the same vertex ($u = v$) and has length greater than zero.
- The path or circuit is said to pass through the vertices $x_1, x_2, \dots, x_{n-1}$ and traverse 遍历 the edges $e_1, \dots, e_n$.
- A path or circuit is simple if it does not contain the same edge more than once.

This terminology is readily extended to directed graphs. (see text)

Paths₃

Example: In the simple graph here:

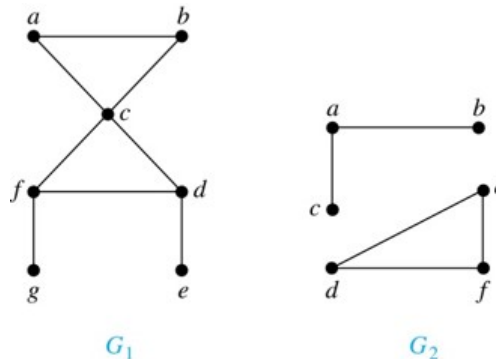


- a, d, c, f, e is a simple path of length 4.
- d, e, c, a is not a path because e is not connected to c .
- b, c, f, e, b is a circuit of length 4.
- a, b, e, d, a, b is a path of length 5, but it is not a simple path.

10.4.3 Connectedness in Undirected Graphs

Definition: An undirected graph is called *connected* 连通的 if there is a path between every pair of vertices. An undirected graph that is not *connected* is called *disconnected*. We say that we *disconnect* a graph when we remove vertices or edges, or both, to produce a disconnected subgraph.

Example: G_1 is connected because there is a path between any pair of its vertices, as can be easily seen. However G_2 is not connected because there is no path between vertices a and f , for example.

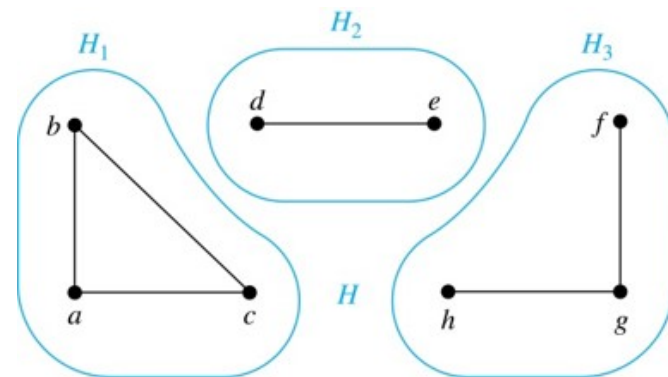


[Jump to long description](#)

Connected Components(opt)

Definition: A *connected component* 联通分支 of a graph G is a connected subgraph of G that is not a proper subgraph of another connected subgraph of G . A graph G that is not connected has two or more connected components that are disjoint and have G as their union.

Example: The graph H is the union of three disjoint subgraphs H_1 , H_2 , and H_3 , none of which are proper subgraphs of a larger connected subgraph of G . These three subgraphs are the connected components of H .



[Jump to long description](#)

10.4.5 Connectedness in Directed Graphs₁

Definition: A directed graph is *strongly connected* if there is a path from a to b and a path from b to a whenever a and b are vertices in the graph.

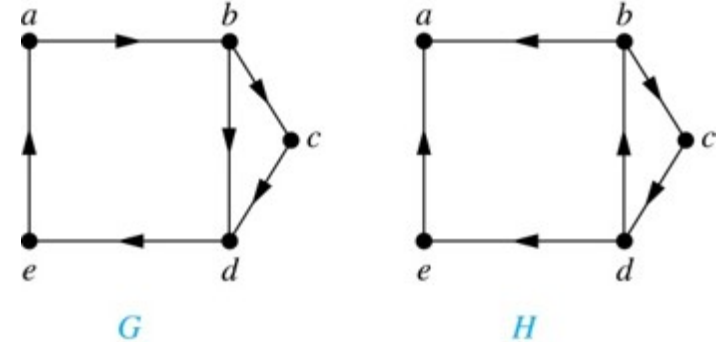
Definition: A directed graph is *weakly connected* if there is a path between every two vertices in the underlying undirected graph, which is the undirected graph obtained by ignoring the directions of the edges of the directed graph.

Connectedness in Directed Graphs₂

Example: G is strongly connected because there is a path between any two vertices in the directed graph.

Hence, G is also weakly connected.

The graph H is not strongly connected, since there is no directed path from a to b , but it is weakly connected.



Definition: The subgraphs of a directed graph G that are strongly connected but not contained in larger strongly connected subgraphs, that is, the maximal strongly connected subgraphs, are called the *strongly connected components* 强联通分支 or *strong components* of G .

Example (continued): The graph H has three strongly connected components, consisting of the vertex a ; the vertex e ; and the subgraph consisting of the vertices b, c, d and edges (b, c) , (c, d) , and (d, b) .

[Jump to long description](#)

Section Summary⁴

1, Paths

Let n be a nonnegative integer and G an undirected graph. A *path* of *length* n from u to v in G is a sequence of n edges $e_1, \dots, e_n$ of G for which there exists a sequence $x_0 = u, x_1, \dots, x_{n-1}, x_n = v$ of vertices such that e_i has, for $i = 1, \dots, n$, the endpoints x_{i-1} and x_i .

When the graph is *simple*, we denote this path by its vertex sequence $x_0, x_1, \dots, x_n$ (since listing the vertices uniquely determines the path).

A path or circuit is *simple* if it does not contain the same edge more than once.

2, Connectedness in Undirected Graphs

An undirected graph is called *connected* 连通的 if there is a path between every pair of vertices. An undirected graph that is not *connected* is called *disconnected*.

Definition: A *connected component* 联通分支 of a graph G is a connected subgraph of G that is not a proper subgraph of another connected subgraph of G .

Section Summary₄

3, Connectedness in Directed Graphs

A directed graph is *strongly connected* if there is a path from a to b and a path from b to a whenever a and b are vertices in the graph.

A directed graph is *weakly connected* if there is a path between every two vertices in the underlying undirected graph

Definition: The subgraphs of a directed graph G that are strongly connected but not contained in larger strongly connected subgraphs, that is, the maximal strongly connected subgraphs, are called the *strongly connected components* 强联通分支 or *strong components* of G .

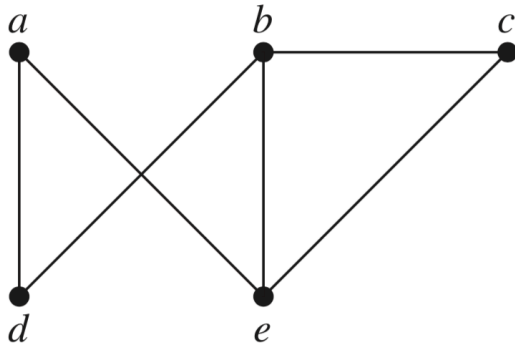
1. Does each of these lists of vertices form a path in the following graph? Which paths are simple? Which are circuits? What are the lengths of those that are paths?

a) a, e, b, c, b

b) a, e, a, d, b, c, a

c) e, b, a, d, b, e

d) c, b, d, a, e, c



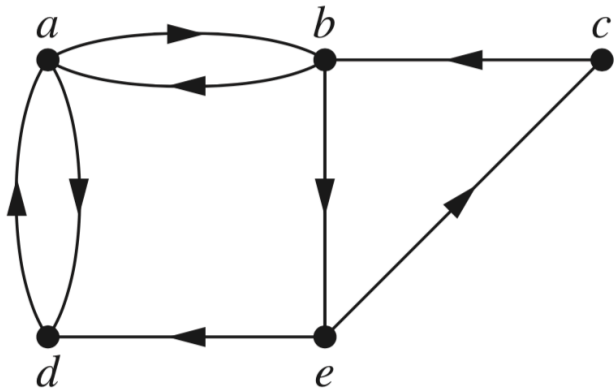
2. Does each of these lists of vertices form a path in the following graph? Which paths are simple? Which are circuits? What are the lengths of those that are paths?

a) a, b, e, c, b

b) a, d, a, d, a

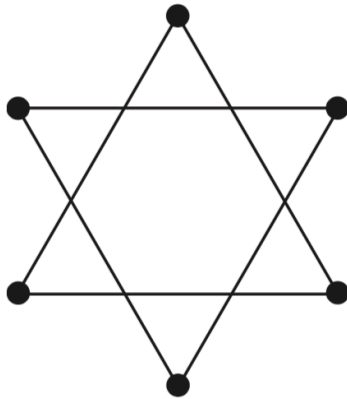
c) a, d, b, e, a

d) a, b, e, c, b, d, a



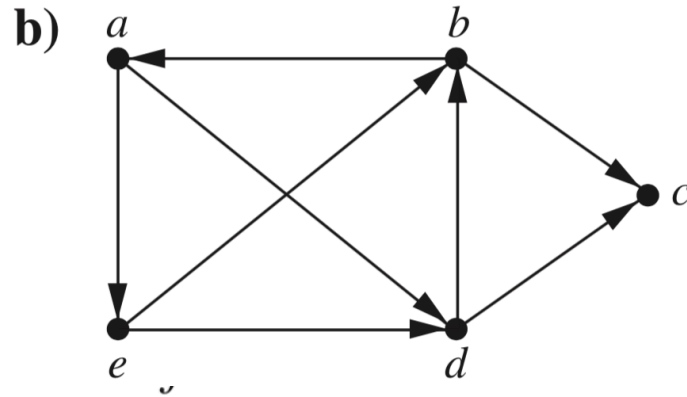
In Exercises 3–5 determine whether the given graph is connected.

5.

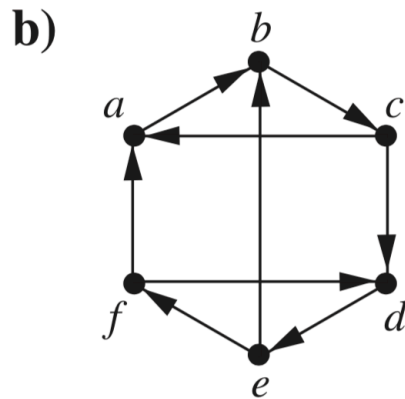


6. How many connected components does each of the graphs in Exercises 3–5 have? For each graph find each of its connected components.

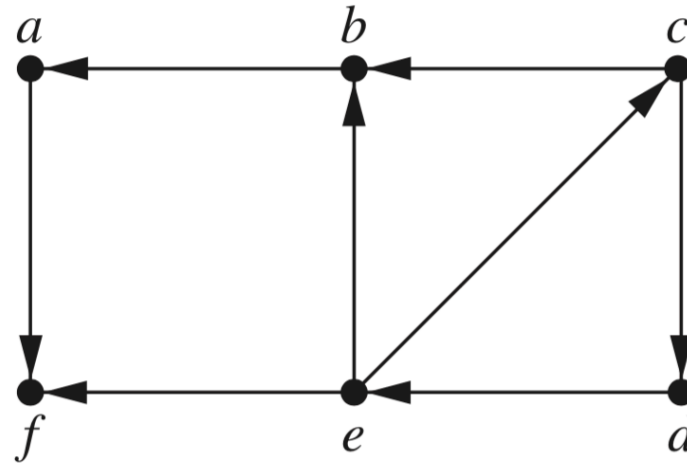
11. Determine whether each of these graphs is strongly connected and if not, whether it is weakly connected.



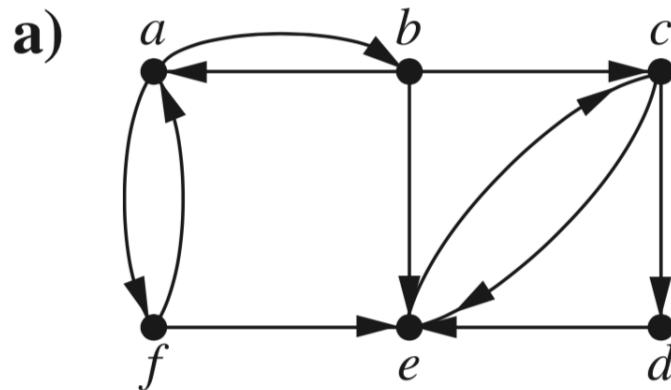
12. Determine whether each of these graphs is strongly connected and if not, whether it is weakly connected.



14. Find the strongly connected components of each of these graphs. **b)**



15. Find the strongly connected components of each of these graphs.



14 Therefore the strongly connected components are $\{a\}$, $\{b\}$, $\{c, d, e\}$, and $\{f\}$.

15 $\{a, b, f\}$, $\{c, d, e\}$